



PETER THE GREAT ST PETERSBURG POLYTECHNIC UNIVERSITY
Institute of Cybersecurity and Computer Science

COURSE PROJECT REPORT

EcoAgents

Artificial Life Ecosystem Simulation with Neural Learning and Evolution

Course Project

Student Mohammad Azimi

Group 5140901/61701

Course Intelligent Systems

Instructor Галина Федоровна Малыхина

Saint Petersburg
2026

Abstract

This report presents EcoAgents, a Java-based artificial-life simulation that models an ecosystem of herbivores, carnivores, and plants. The project implements the central concepts from the course task: a toroidal grid environment, local perception, a sensor-to-brain-to-action agent architecture, a simple neural decision system, winner-takes-all action selection, energy metabolism, food chains, and reproduction with neural inheritance. The implementation extends the base specification with a lightweight reinforcement-style learning rule, exploratory actions, neural mutation, generation tracking, a test suite, and an interactive Swing graphical interface. Controlled tests verify toroidal movement, sensor encoding, neural output selection, eating, learning, reproduction, inheritance, and mutation. In a full deterministic run with seed 42, natural reproduction occurred at iteration 121 and the simulation completed 400 iterations without failure. The result is an interpretable multi-agent artificial-life system in which individual agents adapt during their lifetime while inherited neural parameters allow simple evolutionary change between generations.

Keywords

Artificial life; multi-agent systems; neural networks; reinforcement-based learning; agent-based modeling; evolutionary adaptation; Java Swing; toroidal environment.

Contents

1. Introduction and Task Context	3
2. System Architecture	3
3. Environment and Perception	4
4. Neural Decision System	5
5. Learning and Exploration	6
6. Energy, Feeding, Death, and Reproduction	6
7. Graphical User Interface	7
8. Testing and Validation	9
9. Design Decisions and Limitations	11
10. Possible Future Improvements	11
11. Conclusion	11
Appendix A. Build and Run Commands	11
Appendix B. Main Regression Tests	12
References	12

1. Introduction and Task Context

The course task defines an artificial-life environment in which carnivorous and herbivorous agents live on a grid whose borders are connected as a torus. Each animal has sensors, a processing component (brain), and actions. Animals observe only a local neighborhood, and the local perception is divided into Front, Left, Right, and Nearness areas. The specified action set consists of turning left, turning right, moving forward, and eating. The brain is modeled as a neural network whose output with the largest value selects the action. The task also specifies energy consumption, feeding rewards, death at zero energy, and reproduction when an animal reaches 90% of maximum energy [1].

EcoAgents implements these core requirements as a working Java application and makes several additional engineering choices where the handout does not define exact numerical details. These choices include the geometry of the Nearness region, the learning rule, exploration probability, metabolism rate, mutation parameters, and graphical visualization.

1.1 Project Objectives

- Implement a toroidal artificial-life environment with herbivores, carnivores, and plants.
- Represent each animal as an autonomous sensor-brain-action agent.
- Encode local perception as the 12 named environmental sensor inputs from the task.
- Use a neural decision model with four outputs and winner-takes-all action selection.
- Implement energy, feeding, death, reproduction, neural inheritance, and mutation.
- Allow neural parameters to adapt during an animal's lifetime using reward feedback.
- Provide an interactive GUI that makes internal agent state and ecosystem evolution observable.
- Validate important subsystems with repeatable tests.

2. System Architecture

EcoAgents is organized into six main packages: model, environment, agents, brain, simulation, and ui. This separation keeps low-level data structures, world logic, agent behavior, neural computation, simulation control, and visualization independent and easier to test.

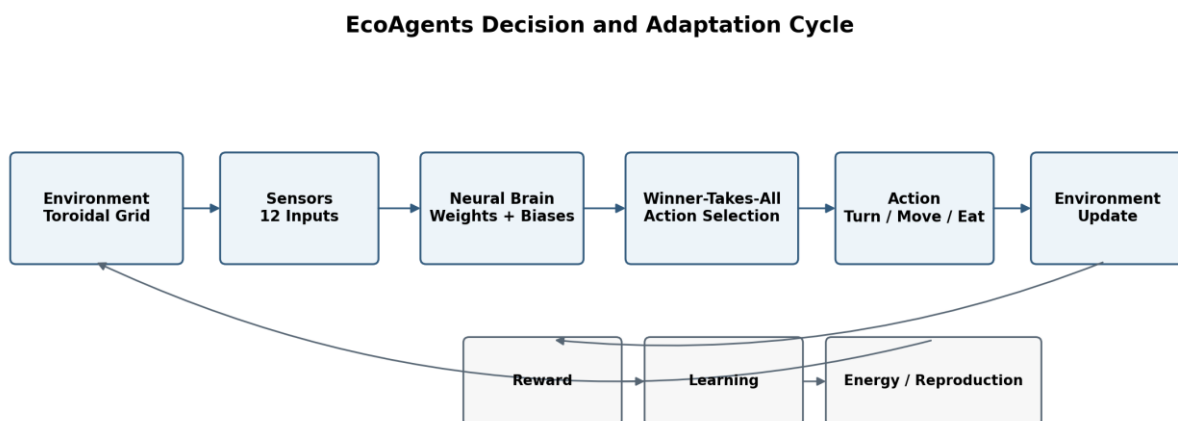
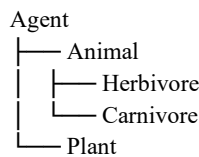


Figure 1. EcoAgents decision and adaptation cycle.

Package	Responsibility
ecoagents.model	Position, Direction, Action, and SensorData data structures.

Package	Responsibility
ecoagents.environment	Toroidal world, occupancy, sensing, movement, and local queries.
ecoagents.agents	Base Agent class and the Animal, Herbivore, Carnivore, and Plant entities.
ecoagents.brain	NeuralBrain, neural outputs, learning updates, inheritance, and mutation.
ecoagents.simulation	Population initialization, iteration loop, reproduction, and statistics.
ecoagents.ui	Swing GUI, grid rendering, agent inspection, charts, and event log.

2.1 Agent Inheritance Structure



Animal contains the common neural, energy, generation, sensing, decision, learning, and reproduction-related state. Herbivore and Carnivore specialize feeding targets and species-specific reward shaping. Plant is a static resource and does not contain a neural brain.

3. Environment and Perception

3.1 Toroidal Grid

The simulation uses a 15×20 grid. Toroidal wrapping means that a coordinate leaving one side of the grid re-enters from the opposite side. This removes hard walls and follows the environment model defined in the course handout [1].

Test movement	Wrapped result
(3, 11) moving EAST	(3, 0)
(0, 5) moving NORTH	(6, 5)
(2, 0) moving WEST	(2, 11)
(6, 8) moving SOUTH	(0, 8)

3.2 Local Sensor Model

Animals cannot see the whole environment. Their perception is encoded into 12 values: the counts of Herbivores, Carnivores, and Plants in the Front, Left, Right, and Nearness regions. The handout tables explicitly name these 12 inputs as HF, CF, PF, HL, CL, PL, HR, CR, PR, HN, CN, and PN [1].

Input	Meaning
HF / CF / PF	Herbivores / Carnivores / Plants in Front

Input	Meaning
HL / CL / PL	Herbivores / Carnivores / Plants on the Left
HR / CR / PR	Herbivores / Carnivores / Plants on the Right
HN / CN / PN	Herbivores / Carnivores / Plants in Nearness

EcoAgents defines Nearness as the eight directly adjacent cells. Front, Left, and Right use cells at distance two and three relative to the current orientation. This geometry is an implementation decision: the task defines the four local areas conceptually but does not prescribe exact cell coordinates.

```

N N N
N A N
N N N

```

A = current animal, N = Nearness cell

4. Neural Decision System

Each animal owns a NeuralBrain with 12 inputs and four outputs. For output i , the implementation evaluates a linear weighted sum with a bias:

$$\text{Out}(i) = \text{bias}(i) + \sum w(i,j) \times u(j)$$

The four outputs map to TURN_LEFT, TURN_RIGHT, MOVE_FORWARD, and EAT. The action with the largest output is selected, implementing the winner-takes-all principle required by the task [1].

Output index	Action
0	TURN_LEFT
1	TURN_RIGHT
2	MOVE_FORWARD
3	EAT

4.1 Example Neural Decision

```

TURN_LEFT  = 0.70
TURN_RIGHT = 0.10
MOVE_FORWARD = 1.50
EAT        = 2.10

```

Selected action: EAT

The corresponding regression test confirms that EAT wins when its output is the largest. Neural parameters are initialized randomly for simulation agents, while deterministic values are used in unit-style tests.

5. Learning and Exploration

The handout states that neural weights are adapted during life but does not specify a concrete adaptation algorithm [1]. EcoAgents therefore uses a simple reinforcement-style rule that updates only the selected action pathway:

$$\text{weight} \leftarrow \text{weight} + \text{learningRate} \times \text{reward} \times \text{input}$$

$$\text{bias} \leftarrow \text{bias} + \text{learningRate} \times \text{reward}$$

The learning rate is 0.05. Neural parameters are clamped to the interval $[-3, 3]$ to avoid uncontrolled growth.

5.1 Reward Design

Situation	Base reward
Successful EAT	+1.00
Failed EAT	-0.10
Successful MOVE_FORWARD	+0.02
Blocked MOVE_FORWARD	-0.05
TURN_LEFT / TURN_RIGHT	-0.01

Species-specific reward shaping provides additional guidance. For example, a Herbivore receives an additional positive signal for choosing EAT when a plant is in Nearness, or MOVE_FORWARD when a plant is detected in Front. Carnivores use the same idea with Herbivores as prey.

5.2 Exploration

During the full simulation, 10% of decisions are exploratory random actions and approximately 90% follow the current neural winner. This prevents an animal from becoming permanently trapped in a poor initial action pattern. Deterministic subsystem tests disable exploration.

6. Energy, Feeding, Death, and Reproduction

Parameter	Value
Maximum animal energy	20.0
Initial animal energy	16.0
Metabolism cost per active iteration	0.02
Herbivore food gain	+1.0 per Plant
Carnivore food gain	+2.0 per Herbivore
Reproduction threshold	90% of maximum energy = 18.0
Offspring initial energy	40% of maximum energy = 8.0
Exploration probability	10%
Mutation probability	10% of parameters
Maximum mutation magnitude	0.10

An animal pays metabolism each active iteration. If energy reaches zero, it dies. Herbivores restore energy by consuming Plants, while Carnivores restore energy by consuming Herbivores, matching the energy relationships described in the task [1].

6.1 Reproduction and Evolution

When an animal reaches 18 energy (90% of the maximum), it may reproduce if an empty neighboring cell exists. The child is created as the same species, receives generation = parent generation + 1, and inherits a copy of the parent's current neural parameters. Small random mutations are then applied. Eight energy units are transferred from the parent to the child.

Parent energy before reproduction: 18
 Child energy: 8
 Parent energy after reproduction: 10
 Parent generation: 0
 Child generation: 1

This mechanism combines within-lifetime adaptation (learning) with between-generation variation (inheritance and mutation). The implementation deliberately keeps this process simple and interpretable.

7. Graphical User Interface

The Swing interface makes the simulation observable and suitable for live demonstration. The main window shows population counters, simulation status, the grid, controls, and two inspection tabs.

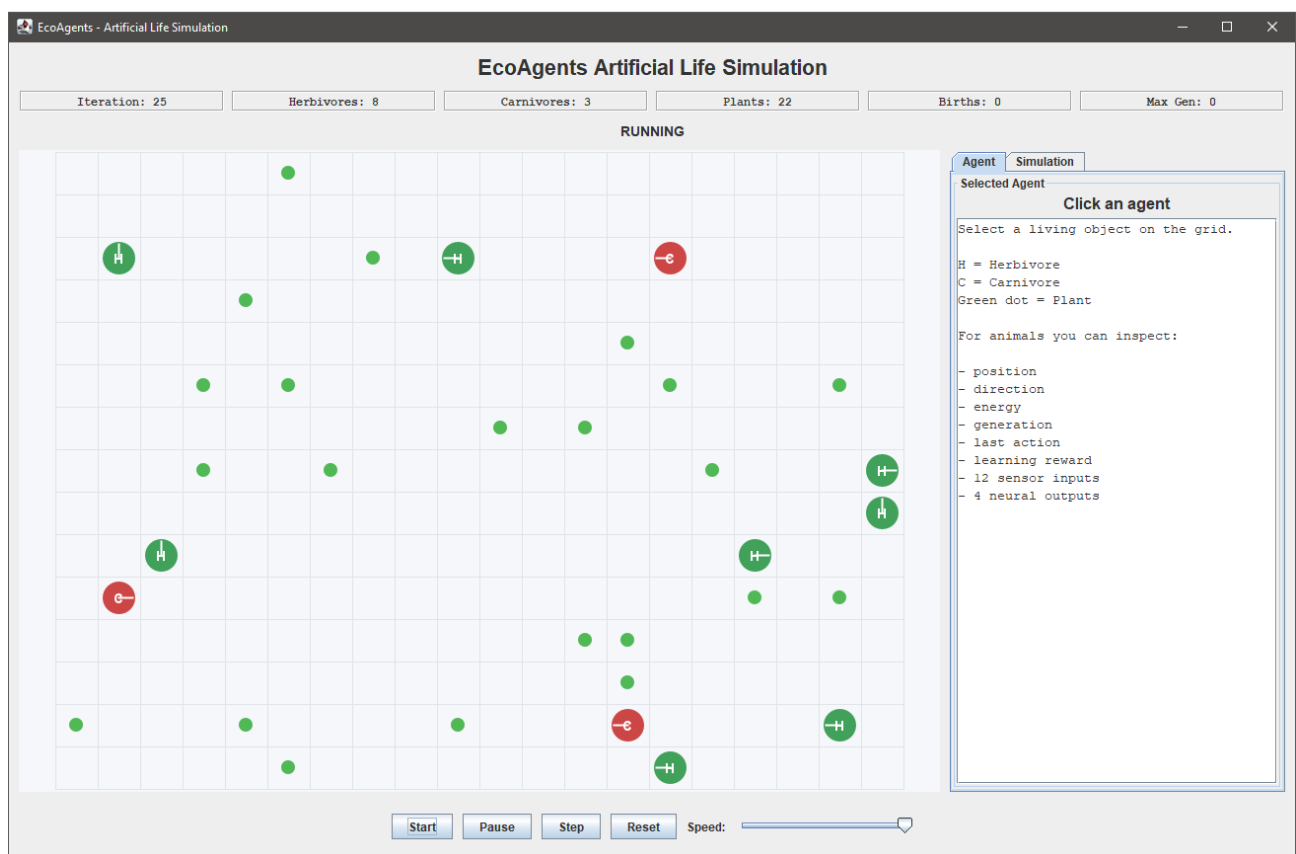


Figure 2. Main EcoAgents interface with the artificial-life grid and Agent tab.

Herbivores are displayed as green circles labeled H, Carnivores as red circles labeled C, and Plants as green dots. A direction marker inside each animal shows its orientation. Clicking a living entity opens its internal state in the Agent inspector.

7.1 Agent Inspector

- type and alive status
- position and direction
- energy and energy percentage
- generation
- last action and chosen action
- learning reward
- 12 sensor inputs
- 4 neural outputs

7.2 Simulation Monitoring

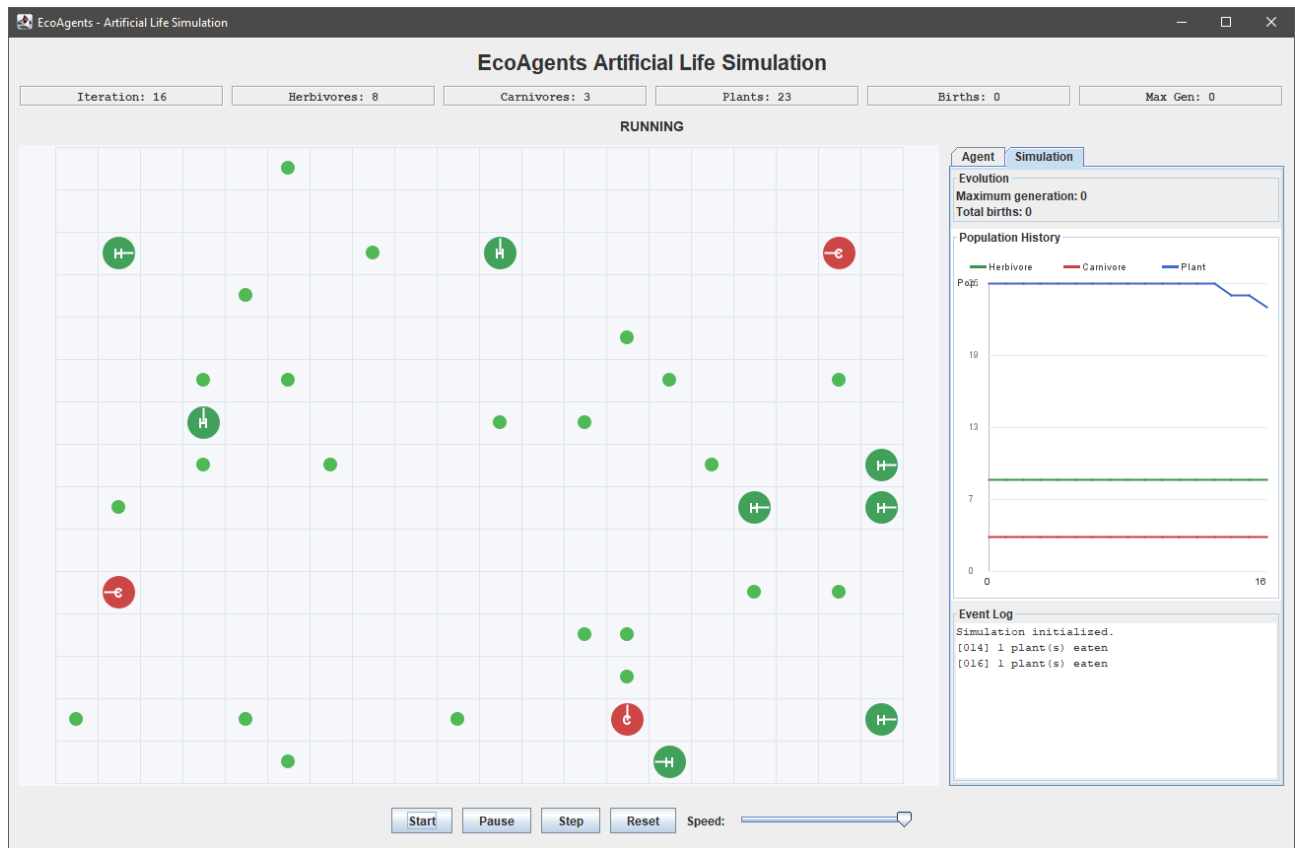


Figure 3. Simulation tab with population history, generation statistics, and event log.

The Simulation tab tracks Herbivore, Carnivore, and Plant populations over time. It also displays total births, maximum generation, and event messages such as plants being eaten, animals being lost, and offspring being born.

8. Testing and Validation

The project uses focused executable regression tests rather than an external testing framework. Each class isolates one important subsystem, and the complete ecosystem test checks that the integrated simulation behaves consistently.

Test	Validated behavior	Result
ToroidalTest	Wrapping on all four grid edges	PASS
DirectionTest	Left/right orientation changes	PASS

Test	Validated behavior	Result
SensorTest	12-input local sensor vector	PASS
NeuralBrainTest	Linear outputs and winner-takes-all action	PASS
AnimalBrainTest	Sense → decide → act → energy update	PASS
LearningTest	Positive reward reinforces successful EAT	PASS
NearnessEatTest	Food can be consumed in 8-cell Nearness	PASS
ReproductionTest	Threshold, inheritance, mutation, generation	PASS
SimulationTest	Multi-agent iteration behavior	PASS
EcosystemBalanceTest	Natural interaction and reproduction over 400 iterations	PASS

8.1 Controlled Learning Result

EAT output before: 1.0000

Learning reward: 1.50

EAT output after: 1.1500

PASS: successful EAT was reinforced.

This confirms that the selected neural pathway changes in the expected direction after a successful food-related action.

8.2 Controlled Reproduction Result

Parent energy before: 18.00 / 20.00

Child generation: 1

Child energy: 8.00 / 20.00

Parent energy after: 10.00 / 20.00

Changed parameters: 7

Maximum mutation: 0.0931

PASS: reproduction, inheritance and mutation work.

8.3 Full Ecosystem Result

A deterministic full simulation using seed 42 began with 8 Herbivores, 3 Carnivores, and 25 Plants. Natural reproduction occurred at iteration 121, producing generation 1. The simulation continued to iteration 400 without crashing.

Iteration	Herbivores	Carnivores	Plants	Births	Max generation
0	8	3	25	0	0
121	6	3	16	1	1

Iteration	Herbivores	Carnivores	Plants	Births	Max generation
200	6	3	10	1	1
400	4	3	2	1	1

The decreasing Plant population demonstrates that resources are being consumed, while the changing Herbivore population reflects both predation and reproduction. Because Plants do not regenerate in the current version, the ecosystem is resource-limited over long runs.

9. Design Decisions and Limitations

9.1 Explicit Implementation Decisions

Several values are not uniquely defined in the task specification. To keep the simulation demonstrable and reproducible, EcoAgents uses the following explicit design decisions:

- Nearness is the eight immediately adjacent cells.
- Front, Left, and Right use a short local range rather than the entire grid.
- Learning uses a simple reward-weight update with learning rate 0.05.
- A 10% exploration probability is enabled in the real simulation.
- Initial energy is 16 and metabolism is 0.02 per active iteration.
- Offspring receive 8 energy and inherited neural parameters are mutated.
- The default run is limited to 400 iterations and uses seed 42.

9.2 Current Limitations

- Plants are finite and do not regenerate, so long simulations eventually become resource-limited.
- The neural model is linear and has no hidden layers.
- The learning rule is a lightweight project-specific reinforcement mechanism, not a full RL algorithm such as Q-learning or policy gradients.
- Reproduction uses mutation only; there is no crossover between two parents.
- Evaluation currently emphasizes functional correctness and demonstrability rather than large-scale statistical experiments across many random seeds.

10. Possible Future Improvements

- Add plant regeneration or a configurable resource growth model.
- Run batches across multiple seeds and export population statistics to CSV.
- Add configurable grid size, population sizes, energy parameters, and mutation settings in the GUI.
- Visualize neural weights and their changes over time.
- Introduce hidden layers or compare alternative neural architectures.
- Compare the current reinforcement-style update with Q-learning or other reinforcement-learning methods.
- Add genetic crossover or adaptive mutation rates for richer evolutionary experiments.
- Save and reload simulations for longer experiments.

11. Conclusion

EcoAgents successfully transforms the artificial-life course specification into a complete interactive Java application. The system implements a toroidal world, local sensors, neural action selection, herbivore and

carnivore feeding, metabolism, death, reproduction, neural inheritance, and mutation. The project also adds an interpretable lifetime learning mechanism and a GUI that exposes both individual agent decisions and population-level dynamics. The regression tests confirm the main subsystems, while the seed-42 ecosystem run demonstrates natural reproduction and a new generation during a complete 400-iteration simulation. The project therefore provides a compact and explainable example of how perception, adaptive decision-making, survival constraints, and evolutionary mechanisms can be combined in an artificial-life multi-agent system.

Appendix A. Build and Run Commands

From Windows CMD, the project can be compiled and launched using:

```
cd /d G:\Projects\Uni-Projects\EcoAgents

rmdir /s /q out
mkdir out

javac -d out src\ecoagents\model\*.java src\ecoagents\brain\*.java src\ecoagents\environment\*.java
src\ecoagents\agents\*.java src\ecoagents\simulation\*.java src\ecoagents\ui\*.java src\ecoagents\*.java

java -cp out ecoagents.Main
```

Appendix B. Main Regression Tests

```
java -cp out ecoagents.ToroidalTest
java -cp out ecoagents.DirectionTest
java -cp out ecoagents.SensorTest
java -cp out ecoagents.NeuralBrainTest
java -cp out ecoagents.AnimalBrainTest
java -cp out ecoagents.LearningTest
java -cp out ecoagents.NearnessEatTest
java -cp out ecoagents.ReproductionTest
java -cp out ecoagents.SimulationTest
java -cp out ecoagents.EcosystemBalanceTest
```

References

- [1] Course handout, “Artificial life modeling,” 6 pages. Defines the toroidal environment, local perception areas, sensor inputs, neural winner-takes-all actions, energy/feeding rules, reproduction threshold, and neural inheritance.
- [2] EcoAgents source repository: <https://github.com/mohammad-azimi/EcoAgents>